



New Client Instance Runbook

Central Intelligence · Ops · Virtual Assistants Philippines · 2026

ONE DEPLOYMENT PER COMPANY

How to stand up a complete Central Intelligence instance for a new company: their own droplet, database, auth project, frontend, and keys. Rehearse this once on a demo instance before doing it for a paying client.

ISOLATION IS THE PRODUCT PROMISE

Never reuse anything from another instance: no API keys, no Supabase project, no Fernet encryption key, no droplet, no Vercel project, no `.env` values. If a value came from another client's instance, it does not belong here.

1. Prerequisites (create these first)

RESOURCE	WHERE	NOTES
Supabase project	supabase.com → New project	Serves BOTH auth and the app DB. Enable the <code>vector</code> extension (Database → Extensions → <code>pgvector</code>). Note the project URL, anon key, JWT secret, and the session-pooler connection string.
DigitalOcean droplet	DO → Create → Droplet	Ubuntu 22.04+, basic tier (\$6/mo is proven adequate). Note the IP.
API domain	Client's DNS or yours	e.g. <code>api.client.com</code> — you'll point an A record at the droplet.
Anthropic + Voyage keys	console.anthropic.com / voyageai.com	Fresh keys per client so spend is attributable. Set budget expectations first.
Vercel project	vercel.com	Created in step 5.
Release tag	this repo	Instances deploy from tags, never from a moving branch.

2. Provision the backend

STEP 2 · PROVISION

One script on a fresh droplet installs Docker, writes this client's env, applies the schema, and starts the full compose stack.

SSH into the fresh droplet as root and run:

```
curl -fsSL0 \
  https://raw.githubusercontent.com/<org>/<repo>/<tag>/scripts/provision_instance.sh
chmod +x provision_instance.sh
./provision_instance.sh https://github.com/<org>/<repo>.git <tag>
```

The script installs Docker, clones the repo at the tag, prompts for this client's values (Supabase URL/keys, DB URL, Anthropic/Voyage keys, domains), auto-generates a fresh `INTEGRATIONS_ENCRYPTION_KEY`, writes `backend/.env` (mode 600), applies the **base schema** (`scripts/apply_base_schema.py` — the alembic chain builds on the SQL in `supabase/migrations/`, so a brand-new database needs those first; idempotent), and starts the compose stack — alembic migrations then run automatically via the one-shot `migrate` service.

NO HOSTED SUPABASE? RUN THE WHOLE THING LOCALLY, FREE

`brew install supabase/tap/supabase`, then from `local-supabase/`: `supabase start`. It runs Postgres (with pgvector), auth, and a Studio UI in Docker and prints the URL/keys. Bootstrap: enable the `vector` extension, run `apply_base_schema` + `alembic upgrade head` against `127.0.0.1:54322`, create the admin via the local auth admin API, seed with `seed_instance`. A ready `backend/.env.local-supabase` template documents the values. Perfect for the demo/testbed instance — zero hosting cost.

- `CLIENT_SYNC_ENABLED` starts **false** — their data source connects later, during onboarding (Phase 4 mapping).
- Re-running the script is safe: it re-checks out the tag and restarts, and never overwrites an existing `.env`.

3. DNS + HTTPS

1. Add an **A record**: `api.client.com → <droplet IP>`.
2. Caddy provisions the Let's Encrypt certificate automatically on first request once DNS resolves (no config needed — the domain comes from `API_DOMAIN` in `.env`).
3. Verify: `curl https://api.client.com/api/v1/health` → 200.

4. Seed the instance

STEP 4 · SEED

Create the client's admin account, then load their business profile from the discovery answers.

1. Create the first admin account in Supabase: Authentication → Users → Add user (email + password for the client's owner/admin).
2. Write a `profile.json` from the discovery answers (see `company2-discovery-checklist.html`):

```
{
  "business_name": "Acme Ltd",
  "vertical": "logistics and freight",
  "business_description": "...",
  "app_name": "Acme Intelligence",
  "tagline": "Operations Command Center",
  "currency_code": "GBP", "currency_symbol": "£",
  "timezone": "Europe/London", "locale": "en-GB"
}
```

3. On the droplet:

```
cd /opt/central-intelligence
docker compose cp profile.json api:/tmp/profile.json
docker compose exec api python -m scripts.seed_instance \
  --profile-json /tmp/profile.json --admin-email owner@client.com
```

5. Frontend (Vercel)

1. New Vercel project from the repo's `frontend/` directory, pinned to the same release tag.
2. Environment variables: `NEXT_PUBLIC_API_URL=https://api.client.com/api/v1` , `NEXT_PUBLIC_WS_URL=wss://api.client.com` , `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` (this client's project).
3. Run `npm run build` locally before deploying — Vercel fails silently on lint. After deploying, confirm the deployment shows **Ready**, not just "queued".
4. Point `app.client.com` at Vercel; add it to `CORS_ORIGINS` in the droplet's `backend/.env` if it wasn't the value given at provisioning (then `docker compose up -d`).

6. Go-live verification

STEP 6 · GO-LIVE

Every box below must be checked before the client gets the URL.

- ☐ `/api/v1/health` 200 over HTTPS
- ☐ Login works with the seeded admin; sidebar shows the client's app name/logo (branding served by `GET /api/v1/config/branding`)
- ☐ Settings → Business shows the seeded profile; saving a change works (admin)

- `docker compose logs beat worker --since 30m` — scheduled tasks no-op cleanly (skipped / zero-count messages, no tracebacks)
- No paid LLM spend in the Anthropic console beyond your own smoke tests (the daily overall-insight synthesis skips itself until real data exists)

KNOWN FRESH-INSTANCE CAVEAT – DEMO SEED ROWS

Four scheduled tasks (`social_stats` linkedin/tiktok, `ads_stats` , `email_stats` , `comments_collector`) upsert hardcoded demo rows so marketing dashboards aren't blank. On a real client instance these will appear as fake campaigns/comments until real integrations connect. Making these seed writers opt-in is a Phase 4 decision — until then, mention it to the client or clear the rows after their integrations go live.

7. What happens next

The instance is live but empty. Data onboarding (Phase 4 of the productization roadmap, [productization-roadmap.html](#)) connects their database: read-only credentials → schema introspection → approved mapping → historical backfill → hourly sync (`CLIENT_DATABASE_URL` + `CLIENT_SYNC_ENABLED=true`). Directors, metrics, and integrations get configured from the discovery answers as part of that phase.

8. Upgrades

```
cd /opt/central-intelligence
git fetch --tags origin && git checkout <new-tag>
docker compose up -d --build      # migrate service applies any new migrations
```

Only deploy tags that passed the staging parity gate ([staging-parity-runbook.html](#)).